

From Spare Compute to Cumulative Mathematics

An open-source strategy for agent-assisted research on hard problems

WILL COOK*

August 2026

ABSTRACT

This paper describes an open-source strategy for agent-assisted mathematical research. The starting point is a practical asymmetry. Building a durable research environment requires navigation, experiment records, formal proof checking, claim boundaries, review tools, and attribution machinery. Once that fixed cost has been paid, another person should not have to rebuild it in order to contribute a mathematical idea or a night of spare compute.

The proposed research commons accepts four independently useful inputs: compute, mathematical direction, infrastructure improvements, and review. Its outputs are not limited to solved problems. Proofs, counterexamples, finite computations, corrected statements, failed mechanisms, formal no-go theorems, and better research tools all reduce the cost or improve the direction of later work. Accepted mathematical returns enlarge a versioned problem corpus; accepted architectural returns improve the process that turns future compute and insight into reviewable mathematics. Provenance and role-specific credit remain attached when a contribution is assimilated.

The implemented case is a public Lean repository organised around eight open Erdős problems. The problems were chosen as difficult research worlds, not because the present system was expected to solve them. A fresh clone contains papers, formal source, explicit open boundaries, corpus queries, validation programs, and structured return paths. All eight problems remain open. The repository is therefore a proof of concept for cumulative research infrastructure, not evidence that distributed agents already outperform mathematicians or that additional compute will solve a named problem.

The paper gives a production model, contribution protocol, trust and security boundary, attribution policy, comparison with volunteer computing, Polymath-style collaboration, formalisation projects, and multi-agent proof systems, and an evaluation agenda. Mathematics is the initial domain because its objects are digital and formal claims can be checked incrementally. Any later extension to experimental science would require domain scientists, physical laboratories, safety governance, and new evidential authorities.

1 The strategy

This project began with one undergraduate working without an institutional research team or a dedicated compute allocation. That fact does not validate the mathematics or the architecture. It makes the strategic question concrete: can the fixed cost of research infrastructure be paid once, published, and then shared by people who bring different scarce inputs?

The intended answer is an open research commons. A mathematician may supply a theorem, counterexample, reference, or correction. A Lean contributor may formalise or audit one statement. A person with spare compute may run an agent against a bounded public frontier. An infrastructure contributor may improve navigation, experiments, validation, reproducibility, or the public contribution path. A reviewer may reconcile a formal statement with its intended meaning or decide that a returned result does

**Authorship and AI use.* Every word of this manuscript was generated by agents based on large language models operating within Will Cook's private research system for artificial intelligence. Cook set the objectives and acceptance criteria, reviewed and authorised the manuscript, and is responsible for its contents. The agents are production tools, not authors. This production disclosure supplies no independent verification; Cook did not independently verify every mathematical claim.

not survive scrutiny. These are different contributions. None should have to masquerade as a solution to receive credit.

The repository is deliberately centred on hard problems that current systems cannot simply dispatch in one prompt. The purpose is to expose the complete research process: selecting a useful subproblem, reading prior work, testing conjectures, recording failed routes, formalising stable steps, explaining the result, and preserving the exact statement that remains open. A solution is one possible output of this process. The cumulative record is the output that can be produced on every serious run.

This is not a proposal to place a public queue in front of a private machine. The public clone must stand on its own. The larger private workbench explains how the initial corpus was produced, but it grants no proof or publication authority to a returned result. Contributors may use any model runner or no model at all. What joins their work is the public problem definition, evidence contract, and review path.

2 A production model for mathematical progress

The phrase “turn compute into mathematics” is useful only after its hidden variables are made explicit. Let

$$C_t, A_t, H_t, I_t, P, K_t, V_t, R_t$$

denote, at time t , available compute, agent capability, contributed human mathematical direction, infrastructure quality, problem selection, accumulated research knowledge, validation capacity, and review capacity. The rate of candidate generation may be written schematically as

$$G_t = G(C_t, A_t, H_t, I_t, P, K_t). \quad (2.1)$$

The rate of *reviewable mathematical progress* is a different quantity:

$$Y_t = Q(G_t, V_t, R_t), \quad (2.2)$$

where Q includes formal checking, statement reconciliation, attribution, and editorial selection. Equations (2.1) and (2.2) are a conceptual production model, not a fitted empirical law. They record complementarity and bottlenecks. More compute can generate more attempts without increasing the rate at which strong results are validated or understood. A better model can still revisit a closed route if the corpus is difficult to navigate. Expert insight can change the search distribution without supplying any compute. Review capacity can be the binding constraint even when proof generation is cheap.

The open-source strategy adds two recursive updates:

$$\begin{aligned} K_{t+1} &= K_t + \Delta_t^{\text{theorem}} + \Delta_t^{\text{counterexample}} + \Delta_t^{\text{no-go}} + \Delta_t^{\text{computation}} + \Delta_t^{\text{correction}}, \\ I_{t+1} &= I_t + \Delta_t^{\text{navigation}} + \Delta_t^{\text{validation}} + \Delta_t^{\text{reproducibility}} + \Delta_t^{\text{governance}}. \end{aligned} \quad (2.3)$$

Only reviewed, scoped returns enter these updates. The first line makes later research less forgetful. The second makes later research less expensive or more reliable. A local failure can therefore have two legitimate effects: it may add a mathematical obstruction to K_t , or reveal a reusable process defect whose repair improves I_t . It cannot change a conjecture’s truth status merely because it changed the process.

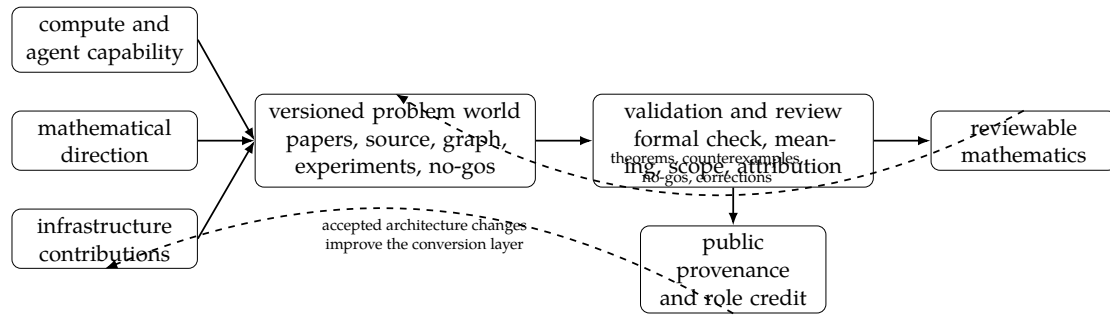


Figure 1: The open-source conversion loop. Search output crosses a validation and review boundary before it becomes public mathematical progress. Accepted mathematical and architectural returns improve different state variables.

The model also gives a longitudinal use for the repository. A fixed, versioned problem world can be revisited as models, runners, and compute budgets change. Comparisons must disclose the starting commit, model, scaffolding, number of attempts, compute, and review procedure. Otherwise a success says little about which variable changed. A public failure record is equally important: capability claims are badly distorted when successes are announced and the number and cost of failed attempts are hidden [1].

3 Why begin with hard mathematics

Mathematics is an unusually clean first domain for this strategy. The objects, programs, papers, and formal statements can all travel in one clone. Many conjectures can be probed by exact computation. A proof assistant can check a stable formal step without waiting for an entire long argument to be written informally and retrofitted later. This does not make mathematical meaning automatic: Lean checks the proposition in the source, not whether it is the proposition the project intended to study.

Computation has a second role. It can provide an artificial form of local intuition to a system that is better at writing and running code than at reproducing a mathematician’s tacit judgement. Exact experiments can expose counterexamples, compare representations, and locate a narrow regularity. Their result updates route selection and conjecture pressure. It does not change the evidence class of an unbounded statement. In Bayesian language, an experiment can update which route deserves attention; it cannot assign a formal posterior probability to a theorem or replace proof.

The eight current problems were selected as demanding research environments, not as a claim that they were the eight most tractable or valuable open problems. Their papers expose materially different contribution points.

Problem	Mathematical world	Examples of exact open contribution points
68	Factorial reciprocal series	Cofinal non-unit carries, weighted collision mass, valuation amplification, endpoint escape, or a non-integral Cramer residual.
243	Reciprocal-tail rigidity	Exclude unbounded divergent-mass negative excursions; prove overlap-height growth, summability, or a variable-rise CRT barrier.
249	Binary totient series	Dyadic phase anti-concentration, carry-rank compression, coupled denominator approximants, or a scoped no-go for one of these routes.
251	Prime-gap dyadic series	Cofinal tail-shift escape or a finite logarithmic-block anti-concentration theorem for actual prime gaps.
257	Mersenne-support subseries	Greedy-orbit recurrence, rational-target classification, fatal-gap exclusion, or realised finite-denominator structure.
269	Three-prime running LCM	Identify the literal rationality-to-carry bridge, prove tail nonintegrality, or construct a faithful two-dimensional representation.
1041	Lemniscate geometry	Repair or refute the saddle decomposition, construct the ray-cut graph, or prove metric gluing and perturbation stability.
1049	Rational-base Lambert series	Build a noncollapsed four-jet or determinant family with analytic decay, or prove a structural obstruction for a named class.

Every row admits more than theorem proving. A mathematician may sharpen a hypothesis or supply a counterexample. A programmer may run a discriminating exact calculation. A formaliser may turn a paper deduction into a checked declaration or find that the two statements disagree. A literature reader may locate a theorem that closes or invalidates a route. The contribution is the exact returned object and its limitation, not the prestige of the problem number.

4 The public research object

The unit of participation is a *problem world*. A problem world includes the endpoint question, current public status, principal theorems, formal source, experiments, known counterexamples, failed mechanisms, source literature, and exact open obligations. A bounded task is a neighbourhood in that world, not an invitation to read the repository indiscriminately.

The current public clone exposes these layers separately:

Surface	What it supplies	What it cannot decide
Problem papers and results map	Mathematical statements, proof spine, limits, and exact open questions	Kernel acceptance or community importance
Lean source and pinned toolchain	Formal propositions and proof terms accepted by Lean	Intended meaning, novelty, or faithful exposition
Corpus queries and navigation graph	Bounded routes through declarations, claims, papers, and relations	Proof or permission to strengthen a claim
Experiments and finite certificates	Exact inputs, programs, finite outputs, and interpretations	An unbounded theorem
Comparator and Palomar	Exact-interface comparison, permitted-axiom checking, a pinned registry record, and an automated editorial filter	Human expert review, novelty, endorsement, or broad acceptance
Research commons	Structured return, review, adoption, correction, provenance, and credit	Automatic correctness or entitlement to merge

The project is a wrapper and composition layer around substantial prior work, not a replacement for it. Its principal inheritances are explicit:

Inherited project or practice	How it is used here
Erdős Problems [7]	Hard questions, historical status, sources, and a problem community become problem-sized formal worlds with exact frontiers and typed return routes.
Lean 4 [8]	Kernel-checked definitions, statements, and proofs are used incrementally inside a search, experiment, exposition, and adoption loop.
DeepMind formal-conjectures [9]	One-problem files, metadata, snapshots, issues, and pull requests are extended with deep path/no-go records and separate mathematical and architecture credit.
Comparator and Palomar [10]	Exact interfaces, permitted-axiom checks, pinned records, and an automated filter receive candidates from a corpus of routes, evidence, and contributor lineage.
Polymath [4]	Open partial and negative collaboration is given replayable evidence bundles and formal, computational, and architectural lineage.
BOINC and GIMPS [2, 3]	Donated compute, executable clients, independent checks, and visible credit inspire mathematician-designed packets for open-ended path mining.

An external runner begins at [the compact agent entry](#). It can inspect all problem frontiers, choose a bounded question, read the relevant paper and source neighbourhood, run experiments or edit formal code, validate the result, and prepare a return. None of these steps requires access to the private workbench. A contributor is free to replace the agent, the scheduler, or the entire search policy while retaining the public evidence boundary.

5 The contribution protocol

The ordinary Git verb is a pull request. A contributor forks or clones the repository, creates a branch, commits a focused change, pushes the branch to a public fork, and opens a pull request. A person who has a result but no patch can open a structured research-progress issue. A person with an infrastructure proposal can use the parallel architecture-proposal path.

The complete protocol has eight steps.

1. **Clone.** Record the exact public commit and environment.
2. **Orient.** Read the claim boundary and the paper governing the selected problem or architecture area.

3. **Select.** Name one bounded frontier, expected evidence, and a stop condition. Working on all problems is allowed, but each returned result remains independently scoped.
4. **Work.** Use any mixture of reasoning, source research, computation, Lean, and human mathematical judgement. Preserve useful negative results instead of rewriting the run as a linear success.
5. **Validate.** Run the checker appropriate to the claim. A computation receives a computation receipt; a formal theorem receives a Lean receipt; neither receives semantic or publication authority from that fact alone.
6. **Return.** Submit source, evidence, starting commit, tool disclosure, collaborators, requested contribution roles, and the exact stronger statement that remains unproved.
7. **Review and adopt.** Reviewers reproduce the evidence, reconcile formal and informal statements, decide scope, and either accept, request revision, reject, or preserve the return as a scoped negative result.
8. **Publish and credit.** An accepted generation is linked to the paths and commit that assimilated it. Later corrections append lineage; they do not erase the earlier contributor.

The accessibility claim is role-bounded. A non-specialist compute contributor is not asked to decide what constitutes a proof or to invent a mathematically meaningful objective unaided. The default work unit is a packet prepared from a mathematician-authored or mathematician-reviewed route: an exact open statement, a relevant source neighbourhood, a permitted experiment or proof obligation, an expected evidence class, and a stopping condition. The contributor and agent execute the packet and return evidence. Lean checks a formal proposition. A qualified reviewer checks whether that proposition says what was intended and whether the return is relevant. The return remains a candidate until the applicable gates have been crossed.

This protocol supports two first-class tracks. The *mathematics track* accepts proofs, reductions, computations, counterexamples, corrections, formalisation, and reproducible failed routes. The *architecture track* accepts improvements to agent workflow, navigation, validation, reproducibility, public experience, governance, and tooling. An architecture return never needs a fictitious problem number, and it cannot request promotion of a mathematical claim.

The distinction between a return and an adopted result is load-bearing. A pull request is a proposed delta. A review act records a decision. A release generation is an immutable public state. Keeping these separate permits credit for a useful counterexample, correction, or design even when no theorem is merged.

6 From an agent claim to mathematical attention

An agent saying “solution found” is an event in a run log, not a project claim. An informal derivation may contain an unnoticed gap. A Lean theorem may be completely kernel-correct while formalising the wrong statement, assuming an inadmissible axiom, or proving a result too weak to settle the problem. Formal verification therefore strengthens one layer of evidence; it does not collapse correctness, intended meaning, novelty, significance, and community acceptance into a single status.

The project uses a deliberately asymmetric attention funnel:

1. **Candidate.** A person or agent returns a scoped argument, computation, formal declaration, counterexample, or no-go.
2. **Replay.** Automated checks reproduce the code and attach the correct evidence class without promoting its claim.
3. **Internal challenge.** Independent agents or contributors try to break the statement, locate stronger prior art, vary the experiment, and reconcile formal and informal formulations.
4. **Maintainer triage.** A human checks whether the bundle is coherent enough, relevant enough, and sufficiently de-risked to justify scarce specialist attention.

5. **External record.** A mature Lean result may be packaged through Comparator and submitted to Palomar; a mathematical account relevant to a listed problem may be placed before the Erdős Problems community and the appropriate research audience.
6. **Acceptance.** Mathematicians inspect, explain, criticise, reuse, publish, or reject the work over time. Broad mathematical acceptance is exogenous to this repository and cannot be granted by its maintainer.

Palomar is valuable precisely because its claim is narrower than peer review. It mechanically verifies a pinned challenge–solution interface using Comparator and kernel checking, records structured disclosure, and applies a documented automated editorial filter. It explicitly does not endorse the result, establish novelty, or provide human expert review [10]. Likewise, the Erdős Problems forum asks long proofs or partial proofs to be linked as external documents and filters implausibly incomplete claims; it is a route to relevant attention, not a substitute for that community’s judgement [7].

Multiple contributors can improve triage, but headcount is not mathematical evidence. What raises a bundle’s priority is costly, legible corroboration: independent replay, adversarial review, distinct contributors repairing different failure modes, a stable formal interface, and an exposition that a specialist can audit. This social proof should allocate attention, never inflate the theorem status. It lets a contributor without institutional connections accumulate a public chain of reasons for an expert to look, without requiring the expert to sift every raw agent transcript.

7 Progress, provenance, and credit

The project recognises progress in evidence classes rather than by counting commits or theorems. A return may contain:

- an unconditional theorem or a formal proof of an existing statement;
- a counterexample or corrected hypothesis;
- a no-go theorem excluding a defined strategy class;
- a bounded computation with code, range, and interpretation;
- a conditional reduction with its antecedent still visible;
- a reproducible failed route that prevents duplicated search;
- a literature correction or exact source locator;
- an architecture proposal or validated infrastructure change; or
- exposition or review that changes what another person can understand and check.

These items should not be placed on one scalar leaderboard. Compute volume, mathematical depth, software quality, correction work, exposition, and review are not commensurable. A public recognition view can instead expose the contributor, accepted artefacts, evidence classes, problem or architecture area, and contribution roles. The current receipt schema supports the fourteen CRediT roles as an optional vocabulary, including conceptualisation, methodology, software, validation, investigation, and writing [13]. CRediT describes work; it does not decide authorship.

The default promise is simple: the project does not demand ownership of a contributor’s underlying idea in exchange for adoption. If a mathematical or architectural contribution is assimilated, the public lineage should continue to name who originated it and what survived. Substantial authorship and formal publication decisions remain separate judgements. A role ledger is more honest than either a winner-takes-all solver credit or a pseudonymous collective record when the project can preserve exact provenance.

This policy is deliberately generous because attribution is part of the growth strategy. A stranger needs a reason to spend thought or compute on a project whose open problems remain unsolved. Durable, inspectable credit for partial and negative work is one such reason. It is also technically useful: provenance makes later correction, reproduction, and literature attribution possible.

8 Distributed compute without distributed authority

Volunteer compute has an established precedent. BOINC packages scientific jobs for heterogeneous consumer devices, and BOINC Central explicitly aims to make volunteer computing available to scientists without the resources to operate their own project [2]. GIMPS shows a mathematical version: volunteers run a common search program, independent machines confirm a prime, and discovery credit includes the compute donor, software authors, server operator, and wider volunteer effort [3].

Agent-assisted research is less uniform than either example. Most tasks are not interchangeable work units with a predetermined verifier. A runner may change code, propose a conjecture, or misread the problem. The public system therefore distributes *search* while keeping authority local to each evidence type. Mathematicians and formalisation contributors design or review the routes; volunteers may execute them with Claude Code, Codex, Cursor, Antigravity, another open or proprietary harness, or no agent at all. The runner is replaceable. The task packet, evidence boundary, and return record are the shared protocol. Returned code is untrusted until reviewed. Expensive or privileged continuous-integration jobs must not execute fork code with repository secrets. In particular, a GitHub `pull_request_target` workflow must not check out and execute an untrusted fork; GitHub documents that combination as a route to secret leakage and repository compromise [14]. Initial checks should run with read-only permissions, no secrets, bounded resources, and explicit artefact retention. Promotion to trusted infrastructure is a later review decision.

A useful compute return records at least the source commit, model and runner, tool disclosure, prompt or task packet, wall-clock and token or hardware budget where available, commands, changed paths, outputs, and validation receipts. It also records failures and the stopping rule. This is required for capability comparisons as well as security: a claimed improvement is not interpretable when attempts, scaffolding, and cost are invisible.

The first public mode may remain deliberately simple: contributors clone the repository and run their own agents locally. A later volunteer-compute layer could distribute signed, immutable task packets and receive result bundles without granting write access. It should be built only after task identity, sandboxing, deduplication, resource budgets, result replay, and abuse handling have explicit owners.

9 Precedents and the missing composition

No part of this strategy is without precedent. The project deliberately packages practices developed by the Erdős Problems project, Lean and mathlib, Comparator and Palomar, DeepMind’s formal-conjectures, Polymath, BOINC, GIMPS, and recent multi-agent formalisation systems. The intended contribution is an accessible composition: a person should be able to enter through one clone without first recreating those infrastructures.

Distributed compute. BOINC and GIMPS show that members of the public will donate hardware to a scientific objective when the client is easy to run, the work is visible, and credit is legible [2, 3]. Their tasks are much more mechanically uniform than open-ended proof research.

Distributed mathematical insight. Polymath projects invite participants at different mathematical levels to share small observations as they occur. Their rules explicitly welcome tentative and negative insights, provided they are made clear enough for others to absorb, and treat the project as collaboration rather than a race [4]. Polymath supplies a social protocol; it does not supply a formal proof, computation, and agent-return substrate for every comment.

Distributed formalisation. Mathlib uses ordinary fork-and-pull-request practice, human review, and source-level attribution. The Carleson formalisation used a public blueprint and many claimable lemma-sized tasks, with formalisation feeding corrections back into the informal plan [5, 6]. Google DeepMind’s formal-conjectures repository similarly uses one-problem files, issue assignment, pull requests, metadata, and stable snapshots for a growing Lean benchmark [9]. These projects demonstrate that large checked mathematical objects can be made publicly divisible.

Multi-agent formal research. Agent Hunt studies bounties, locks, guarded ownership, and collaborative agents for autoformalisation [11]. Lean Atlas uses formal dependency information to reduce the

declarations a person must inspect for semantic verification [12]. These mechanisms address search, coordination, and review focus. They do not by themselves provide a public commons in which mathematics and improvements to the research architecture enter one provenance-preserving adoption path.

Proof abundance. Tao separates problem solving into generation, verification, exposition, digestion and acceptance, and canonicalisation, and argues that proof abundance will create bottlenecks between these stages [1]. The strategy here treats those bottlenecks as contribution surfaces. A reviewer who prevents an overclaim or an expositor who makes a hard step recoverable is not ancillary to the research pipeline.

The proposed composition joins these precedents. It combines volunteer compute, small shared insights, formal task decomposition, agent-native problem worlds, explicit negative knowledge, untrusted public returns, role-aware credit, and human review. The candidate contribution is this composition and its implementation in one open mathematical corpus. The paper makes no universal priority claim and reports no controlled comparison showing that the composition increases discovery rate.

10 Participation and growth

The project should present several independent reasons to clone the repository.

- A **mathematician** can see eight exact frontiers and contribute a lemma, construction, counterexample, reference, or critique without learning the private orchestration system.
- A **Lean user** can formalise a paper deduction, audit a statement, simplify a proof, or improve the checked interface.
- A **compute hobbyist** can run a local agent against a real open problem using a bounded route designed or reviewed by mathematicians, and return reproducible evidence even when it is negative. The contributor is not asked to certify a proof claim.
- An **agent builder** can compare runners on a stable problem world while disclosing compute, attempts, and starting state.
- An **infrastructure contributor** can improve the conversion from all other inputs to useful mathematical output.
- A **reviewer or expositor** can reconcile meaning, rank results, correct attribution, or turn a checked proof into recoverable understanding.

The lowest-friction invitation should appear on the first screen of the README: clone the project, point a local agent at the compact entry file, and ask it to choose a bounded frontier that matches the available tools. The same screen must say that all eight problems remain open, that a useful return need not solve one, and that architecture contributions receive first-class credit.

Publicity should follow a working contribution loop rather than precede it. A Hacker News launch, research talk, or model-community post can then make a specific claim: a stranger can clone the corpus, reach an exact open question, run or improve the machinery, return a typed result, and see accepted work in public lineage. Stars, clones, and raw agent-hours measure attention or activity. They do not measure mathematical progress.

11 Evaluation

The strategy should be evaluated at each conversion boundary. Useful initial measures include:

- time from a cold clone to a correctly scoped first task;
- fraction of returned bundles that can be replayed at the stated commit;
- fraction requiring statement or evidence-class correction;

- time from return to first substantive review and to adoption decision;
- repeated-dead-end rate before and after a no-go enters the corpus;
- reviewer time per accepted result family;
- contribution diversity across mathematics, computation, software, validation, exposition, and review;
- correction lineage completeness and attribution retention; and
- change in useful output at controlled compute when navigation or another infrastructure component changes.

The numerator must remain claim-bounded. One accepted counterexample may be more useful than hundreds of generated lemmas. Generated certificate shards must not be counted as independent discoveries. A theorem accepted by Lean but rejected on intended meaning is not a successful public transition. Likewise, a strong negative result can improve the corpus even though the endpoint problem remains open.

Longitudinal model comparisons should use immutable problem snapshots and held-out variants where possible. Public hard problems are susceptible to training-data contamination, and the repository itself will become more informative over time. A later model therefore receives both a stronger model and a richer corpus. The design should measure these factors separately rather than presenting every later success as model improvement.

The present work supplies no such evaluation. It establishes an implemented case, an explicit protocol, and falsifiable measures for a later study.

12 Learning from every run

The research memory has two levels. Problem-local memory records the exact objects that change the mathematical search: proved lemmas, counterexamples, finite data, failed mechanisms, and unresolved obligations. General process memory records lessons that transfer: a navigation failure, an experiment pattern, a formalisation pitfall, a validation gap, or a review rule.

The system calls the second movement *up-propagation*. A local lesson is not copied immediately into a universal mathematical-reasoning rule. It is first stated with its evidence, sibling cases, proposed scope, and over-generalisation guard. If the pattern survives review, it may improve a general skill or route. Computational reasoning can likewise become a specialised skill with reusable experiment forms, but every new experiment still records its finite domain and interpretation.

This produces a graph that is richer than a theorem library. Nodes may be endpoints, conjectures, representations, computations, proofs, counterexamples, no-go theorems, reviews, or architecture changes. Edges distinguish proof dependency, implication, equivalence, refutation, obstruction of a method, experimental support, formalisation, correction, and publication. The edge type matters. A model-generated analogy can nominate a route; it cannot become a proof dependency or a public implication without the corresponding evidence.

As this graph grows, it may support a different kind of agent training or retrieval: not only examples of successful proofs, but tempting arguments paired with the exact reason they fail; not only theorem statements, but frontier neighbourhoods showing the remaining cut. Whether this improves mathematical originality or merely makes models more confident within known territory is an open empirical question.

13 A bounded transfer hypothesis beyond mathematics

The same abstract loop appears in experimental science: formulate a hypothesis, design an experiment, observe an outcome, update the theory, and retain both successful and failed interventions. A future agent might define a simulation or experimental script whose variables are linked to a formal model and a semantic account of the hypothesis. Physical observations could then update the research graph and select the next experiment.

Mathematics is only a proof of concept for the *workflow shape*. It is not evidence that the current system can conduct physics, chemistry, or biology. Formal verification has a special strength in mathematics because the target objects and proof rules are digital. In an experimental domain, the instrument, calibration, sample, protocol, measurement uncertainty, physical environment, and causal interpretation become independent authorities. A formally verified control program does not validate the scientific hypothesis or make a laboratory safe.

Any transfer would therefore require domain scientists, controlled physical infrastructure, ethics and safety review, material and environmental limits, and explicit human authority over experiment selection and execution. The initial scope should be low-risk simulation or analysis of existing public data. Autonomous physical experimentation is not a present project goal.

14 Limits and governance

The present strategy has substantial limits.

First, all eight endpoint problems remain open. The repository contains intermediate mathematics and deep records of failed routes, but no evidence that a distributed run will solve any problem. The claim that infrastructure improves the conversion from compute to mathematics has not yet been measured in a controlled experiment.

Second, the project remains maintainer-centred. The initial problem selection, architecture, claim boundaries, review decisions, and public presentation were all chosen through one operator-controlled process. Open source makes those choices inspectable and contestable; it does not make them independent. A serious growth phase needs external maintainers and reviewers, published conflict and appeal rules, and a way for contributor-controlled reports to remain visible when the project declines to adopt them.

Third, review is scarce. More public agents may worsen the burden by generating plausible but low-value returns. Entry tests, evidence schemas, automated replay, and Palomar-style selection should protect expert attention, but automatic filters cannot award significance or community acceptance.

Fourth, credit is contestable. A role vocabulary preserves more information than a single score, but it cannot mechanically decide authorship, priority, or the relative weight of an idea and its proof. The project must disclose its policy in advance and preserve correction history.

Fifth, an open corpus can be gamed. Contributors may optimise for visible activity, generated theorem counts, or a public model comparison. Strong models may have encountered repository text during training. Security bugs may arrive as apparently useful workflow improvements. No automatic leaderboard should be allowed to become the objective function.

Finally, openness does not remove resource inequality. Compute donors, frontier-model providers, established mathematicians, and maintainers have different access and influence. The strategy can lower the fixed cost of entry and make attribution more durable. It cannot by itself make research participation equitable.

15 Execution order

The strategy should be implemented in the following order.

1. Make the README state the experiment, contributor types, exact open boundary, clone command, agent prompt, return paths, and credit policy.
2. Keep every problem world independently navigable from a cold clone, with one paper-level open section and bounded query routes.
3. Accept both mathematical and architecture returns through typed schemas, human review, immutable generations, and public provenance views.
4. Provide runner-neutral local instructions and safe, unprivileged checks before building a hosted or volunteer-compute scheduler.
5. Recruit external reviewers and maintainers before increasing agent throughput substantially.

6. Run controlled studies of navigation changes, model changes, and compute changes against immutable snapshots.
7. Only after these boundaries work should the project consider a public task market, donated compute service, or transfer to another scientific domain.

The first three steps are partly implemented in the current public clone. Turnkey multi-provider mining, a public volunteer-compute scheduler, independent governance, and a cross-domain laboratory interface are not reported capabilities.

16 Conclusion

The open-source strategy is to make mathematical infrastructure a shared conversion layer. A contributor may add compute, mathematical direction, formalisation, software, review, or exposition without rebuilding the whole system and without pretending to have solved an endpoint problem. Accepted mathematical work enlarges the reusable corpus. Accepted architecture work improves the process that produces and checks later work. Both retain public provenance.

The eight Erdős problems are hard test worlds for this design. They make failure unavoidable enough that the system must learn to preserve it, and deep enough that improvements in models, compute, infrastructure, and human insight can be observed over time. Their role is not to guarantee a dramatic solution. It is to ensure that the research commons is exercised on real mathematics with exact open boundaries.

The strongest claim at present is therefore architectural. A fresh clone can carry a stranger from a hard question to a bounded frontier and give that stranger a path for returning evidence and receiving credit. The next test is social and empirical: whether outside contributors can use that path, whether reviewers can absorb the returns, and whether the accumulated graph makes the next attempt genuinely better.

A Public entry routes

The public repository is [wcook04/plectis-lean-erdos249-257](#). The shortest current routes are:

Question	Public route
What is the experiment?	README.md
Where should an agent begin?	AGENTS.override.md
What is proved and what remains open?	docs/RESULTS.md
Which papers and problems exist?	docs/papers/README.md
How can I contribute mathematics?	CONTRIBUTING.md and the research-progress issue form
How can I improve the architecture?	architecture contribution guide and the architecture-proposal issue form
How is a return validated?	research-commons protocol
How is credit recorded?	credit policy
What do Comparator and Palomar establish?	Comparator guide and Palomar qualification

Declaration of generative AI use

Every word of this manuscript was generated by agents based on large language models operating within Will Cook's private research system for artificial intelligence. The formal proofs and repository software were likewise drafted and revised by the agents through that system under Cook's direction. Cook set the objectives and acceptance criteria, selected and reviewed the public claims, and approved the published version. Cook assumes responsibility for the accuracy, interpretation, and presentation of the work. Generative systems are production tools, not authors, and supply no independent authority.

References

- [1] T. Tao, *Mathematics in the age of AI*, 2026, [arXiv:2608.16753](#).
- [2] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, *Journal of Grid Computing* 18 (2020), 99–122, [DOI](#).
- [3] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, [mersenne.org](#), accessed August 2026.
- [4] Polymath Project, *General polymath rules*, [project rules](#), accessed August 2026.
- [5] Lean community, *Contributing to mathlib*, [contributor guide](#), accessed August 2026.
- [6] L. Becker et al., *A Blueprint for the Formalization of Carleson’s Theorem on Convergence of Fourier Series*, 2025, [arXiv:2405.06423](#).
- [7] T. F. Bloom, *Erdős Problems*, problem database, sources, and forum, [erdosproblems.com](#), accessed August 2026.
- [8] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, *Automated Deduction—CADE 28* (2021), 625–635, [DOI](#).
- [9] Google DeepMind, *formal-conjectures*, [software repository](#), accessed August 2026.
- [10] Palomar Registry, *About Palomar and Contribution policy*, [registry documentation](#) and [submission standard](#), accessed August 2026.
- [11] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, [arXiv:2603.06737](#).
- [12] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [13] NISO, *CRedit: Contributor Roles Taxonomy*, [role definitions](#), accessed August 2026.
- [14] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, [documentation](#), accessed August 2026.